

JAVA

JavaFX

Interfaces Gráficas em Java

Do primeiro Hello World à uma aplicação de bilheteira completa

Apostila completa sobre JavaFX, com layouts, controlos, eventos, persistência de dados e estilização, tudo com exemplos práticos.

CONTEÚDO DA APOSTILA

- 01 JavaFX e o Ambiente de Desenvolvimento
- 02 Anatomia de uma Aplicação JavaFX
- 03 Layouts: GridPane
- 04 Layouts: HBox, VBox, FlowPane e TilePane
- 05 Layouts: BorderPane e StackPane
- 06 Controlos Button e Label
- 07 Seleção Única e Múltipla
- 08 Campos de Entrada
- 09 TitledPane e Accordion
- 10 Tratamento de Eventos
- 11 A Thread da Interface Gráfica
- 12 Modelo de Dados e ListView
- 13 Persistência em Ficheiro de Texto
- 14 Ordenação e Filtragem
- 15 Estilização com CSS
- 16 Transformações, Efeitos e Diálogos

01

JavaFX e o Ambiente de Desenvolvimento

O que é o JavaFX, por que substituiu o Swing e como preparar o projeto no JDK 11+

Do Swing ao JavaFX

Aplicações Java com interface gráfica precisam de botões, listas, caixas de verificação, menus e outros controlos com que o utilizador possa interagir. O JavaFX é o conjunto de APIs que o Java oferece hoje para construir esse tipo de interface — um conjunto de pacotes que sucedeu o Swing, o kit de ferramentas gráfico que dominou o Java durante muitos anos. Com o JavaFX é possível construir interfaces para aplicações de ambiente de trabalho, para a web e para dispositivos móveis, mas o foco desta apostila é a aplicação desktop tradicional.

Um detalhe importante: a partir do JDK 11, o JavaFX deixou de vir embutido no próprio JDK. Isso significa que o SDK do JavaFX precisa de ser descarregado à parte, a partir de openjfx.io, e configurado como uma biblioteca do projeto. Em JDKs anteriores ao 11 esse passo não é necessário, já que o JavaFX ainda fazia parte do núcleo da linguagem.

DICA — Configurando o SDK no IntelliJ

Depois de descarregar e extrair o SDK do JavaFX, crie uma biblioteca global em File → Project Structure → Global Libraries, apontando para a subpasta **lib** do SDK. Isso evita repetir a configuração a cada novo projeto.

O módulo module-info.java

Como o JavaFX passou a ser um conjunto de módulos externos ao JDK 11, projetos que usam o sistema de módulos do Java (JPMS) precisam declarar explicitamente essa dependência. Isso é feito num ficheiro **module-info.java** na raiz do código-fonte:

```
module cineticket {  
    requires javafx.fxml;  
    requires javafx.controls;  
  
    opens cineticket;  
}
```

A cláusula **requires** declara que o módulo depende dos pacotes **javafx.fxml** e **javafx.controls**. Já **opens** permite que o motor de execução do JavaFX aceda, por reflexão, às classes do pacote **cineticket** — algo necessário para que o carregador de FXML consiga instanciar o controlador da tela.

O projeto CineTicket

Ao longo desta apostila vamos construir, passo a passo, uma pequena aplicação de bilheteira de cinema chamada CineTicket: uma lista de sessões de filmes, com data, sala e horário, onde o utilizador pode consultar detalhes, filtrar e ordenar sessões, e reservar bilhetes. Cada capítulo acrescenta uma peça nova a essa aplicação.

02

Anatomia de uma Aplicação JavaFX

Application, Stage, Scene e o padrão MVC que organiza a interface

Application, Stage e Scene

Toda aplicação JavaFX tem uma classe de entrada que estende **Application**, a classe que gere o ciclo de vida do programa. Essa classe é responsável por três métodos principais: **init**, executado antes de qualquer coisa aparecer na tela; **start**, onde a interface é efetivamente construída; e **stop**, executado quando a aplicação está a ser encerrada — um bom lugar para gravar dados antes de fechar.

```
public class MainApp extends Application {  
  
    @Override  
    public void start(Stage stage) throws Exception {  
        Parent root = FXMLLoader.load(getClass().getResource("main.fxml"));  
        Scene scene = new Scene(root, 900, 550);  
  
        stage.setTitle("CineTicket");  
        stage.setScene(scene);  
        stage.show();  
    }  
  
    public static void main(String[] args) {  
        launch(args);  
    }  
}
```

Quando o programa é executado, **main** chama **Application.launch**, que inicia o motor do JavaFX e, logo a seguir, invoca **start** passando um **Stage** já construído pelo runtime. Um **Stage** é o contentor de nível mais alto — a janela em si — e estende a classe **Window**. É possível criar mais do que um **Stage**, mas a grande maioria das aplicações usa apenas um: multiplicar janelas de topo tende a prejudicar a experiência do utilizador.

A metáfora do teatro: Stage e Scene

Quem criou o JavaFX inspirou-se deliberadamente numa metáfora teatral. Um **Stage** (palco) pode receber diferentes **Scene** (cenários) ao longo do tempo — tal como um palco de teatro muda de cenário entre atos sem deixar de ser o mesmo palco. Cada **Scene** é sustentada por um grafo de cena: uma árvore onde cada nó corresponde a um controlo da interface ou a uma forma gráfica. Trocar o que aparece na janela é, na prática, trocar a cena que o palco exhibe com **stage.setScene(novaCena)** — não é preciso reconstruir a janela.

O padrão MVC no JavaFX

O JavaFX foi desenhado tendo em mente o padrão MVC (model-view-controller), que separa o código que manipula dados do código que trata da interface. Na prática: o modelo é a estrutura de dados da aplicação; a vista é o ficheiro FXML, uma variante de XML que descreve a disposição dos controlos; e o controlador é a classe Java que reage às ações do utilizador. O JavaFX não obriga a usar esse padrão, mas segui-lo evita misturar lógica de interface com lógica de negócio na mesma classe.

```
<!-- main.fxml -->
<GridPane fx:controller="cineticket.MainController"
    xmlns:fx="http://javafx.com/fxml"
    hgap="10" vgap="10" alignment="CENTER">
    <!-- controlos entram aqui -->
</GridPane>
```

O atributo **fx:controller** indica ao runtime qual classe Java vai tratar os eventos dessa tela. Sempre que uma propriedade do FXML precisar de ser lida ou alterada em código, associa-se um identificador com **fx:id**, e a variável correspondente no controlador é anotada com **@FXML** — veremos isso em detalhe no capítulo sobre eventos.

03

Layouts: GridPane

Organizando controles em linhas e colunas flexíveis

Organizando filhos numa grelha

O JavaFX disponibiliza oito layouts prontos: **GridPane**, **AnchorPane**, **StackPane**, **HBox**, **VBox**, **FlowPane**, **TilePane** e **BorderPane**. Cada um resolve o posicionamento e o redimensionamento dos seus filhos de uma forma diferente, o que evita ter de escrever manualmente coordenadas x/y para cada controlo. Começamos pelo **GridPane**, provavelmente o mais usado como layout de nível superior.

O **GridPane** organiza os seus filhos em linhas e colunas flexíveis. Cada posição chama-se célula; por padrão, cada linha é tão alta quanto o controlo mais alto que ela contém, e cada coluna é tão larga quanto o controlo mais largo. A posição de cada filho é indicada pelas propriedades anexas **GridPane.rowIndex** e **GridPane.columnIndex**, ambas contadas a partir de zero:

```
<GridPane hgap="10" vgap="10" alignment="CENTER">
  <padding><Insets top="10" bottom="10" left="10" right="10"/></padding>

  <Button text="Confirmar" GridPane.rowIndex="0" GridPane.columnIndex="0"/>
  <Button text="Cancelar"  GridPane.rowIndex="0" GridPane.columnIndex="1"/>
  <Button text="Ajuda"     GridPane.rowIndex="1" GridPane.columnIndex="0"/>
</GridPane>
```

Se dois controlos não receberem **rowIndex/columnIndex**, ambos caem na posição padrão linha 0, coluna 0 — e ficam sobrepostos. Também não é necessário declarar as células em ordem: o **GridPane** posiciona cada filho de acordo com os índices informados, independentemente da ordem em que aparecem no FXML.

Ocupando várias células

Um controlo pode estender-se por mais de uma linha ou coluna com as propriedades **GridPane.columnSpan** e **GridPane.rowSpan**. Isso é útil, por exemplo, para um título ou uma barra de botões que deve ocupar toda a largura da grelha:

```
<Label text="Sessões de hoje" GridPane.rowIndex="0" GridPane.columnIndex="0"
      GridPane.columnSpan="2" GridPane.halignment="CENTER"/>
```

As propriedades **GridPane.halignment** e **GridPane.valignment**, aplicadas a um filho individual, controlam onde esse controlo fica posicionado dentro da própria célula, quando ela é maior do que o tamanho preferido do controlo.

Larguras fixas com ColumnConstraints

Por padrão, cada coluna tem exatamente a largura do seu controlo mais largo. Quando se quer que as colunas tenham larguras proporcionais fixas, usa-se o elemento **columnConstraints**, com um **ColumnConstraints** por coluna, na mesma ordem das colunas:

```
<GridPane hgap="10" vgap="10">
  <columnConstraints>
    <ColumnConstraints percentWidth="60"/>
    <ColumnConstraints percentWidth="40"/>
  </columnConstraints>
  ...
</GridPane>
```

DICA — Depurando o layout

Quando a disposição de um **GridPane** não sai como esperado, ative **gridLinesVisible="true"**. As linhas da grelha ficam visíveis na tela, o que facilita perceber por que um controlo está a ocupar mais ou menos espaço do que o previsto. Lembre-se de remover essa linha antes de entregar a aplicação.

04

Layouts: HBox, VBox, FlowPane e TilePane

Disposições lineares e disposições que se reorganizam sozinhas

HBox e VBox: uma única fileira

HBox dispõe os seus filhos horizontalmente numa única linha, e **VBox** faz o mesmo na vertical. Ambos dimensionam os filhos para a respetiva largura ou altura preferida e, se sobrar espaço, esticam a si próprios em vez de esticar os filhos — a menos que a propriedade **fillHeight** (no **HBox**) ou **fillWidth** (no **VBox**) diga o contrário. É por isso que **HBox** e **VBox** são a escolha natural para uma fileira de botões numa caixa de diálogo:

```
<HBox spacing="10" alignment="BOTTOM_RIGHT">
  <padding><Insets bottom="10" right="10"/></padding>
  <Button text="Confirmar" prefWidth="90"/>
  <Button text="Cancelar" prefWidth="90"/>
</HBox>
```

A propriedade **spacing** define o espaço entre os filhos, e **alignment** controla onde eles ficam dentro da caixa quando ela é maior do que o conteúdo. Diferente do **HBox** e **VBox**, se não houver espaço suficiente, os filhos simplesmente são cortados: nenhum dos dois reorganiza o conteúdo automaticamente.

FlowPane: quando o número de filhos varia

O **FlowPane** é semelhante ao **HBox/VBox**, mas resolve exatamente o problema do corte: quando não há espaço para mostrar todos os filhos numa única linha (ou coluna, se a orientação for vertical), ele automaticamente os envolve para a linha seguinte, e volta a juntá-los se a janela for redimensionada para maior. É a escolha certa quando não se sabe de antemão quantos filhos serão adicionados — por exemplo, ao gerar um botão por sessão de cinema a partir dos dados lidos de um ficheiro:

```
<FlowPane hgap="8" vgap="8" orientation="HORIZONTAL">
  <Button text="Sessão 14:00 – Sala 1"/>
  <Button text="Sessão 16:30 – Sala 2"/>
  <Button text="Sessão 19:00 – Sala 1"/>
  <!-- ... quantos botões forem necessários -->
</FlowPane>
```

TilePane: células de tamanho uniforme

O **TilePane** funciona de forma parecida ao **FlowPane** — também envolve os filhos quando necessário — mas com uma diferença importante: todas as células ('tiles') recebem exatamente a mesma largura, a do filho mais largo. Isso costuma criar um resultado mais organizado visualmente quando os itens representam a mesma categoria de informação, como um mosaico de cartazes de filmes.

Layout	Organiza como	Reorganiza automaticamente?
HBox / VBox	Linha única / coluna única	Não — os filhos são cortados
FlowPane	Linha/coluna que envolve	Sim, ao faltar espaço
TilePane	Mosaico de células iguais	Sim, ao faltar espaço

DICA — Quando usar cada um

Prefira **HBox/VBox** para grupos pequenos e fixos, como os botões de uma caixa de diálogo. Reserve **FlowPane** ou **TilePane** para coleções cujo tamanho varia em tempo de execução, como uma lista de sessões carregada de um ficheiro ou de uma base de dados.

05

Layouts: BorderPane e StackPane

A disposição clássica de janela e o empilhamento de camadas

BorderPane: a disposição clássica de janela

O **BorderPane** é, provavelmente, o layout de nível superior mais usado em aplicações reais. Ele reparte o espaço em cinco posições: topo, base, esquerda, direita e centro. É o esqueleto típico de uma aplicação cliente — um menu ou barra de ferramentas no topo, uma lista de navegação à esquerda, uma área de trabalho ao centro, e uma barra de estado na base.

```
<BorderPane fx:controller="cineticket.MainController"
  xmlns:fx="http://javafx.com/fxml">
  <top>
    <ToolBar><Button text="Abrir"/></ToolBar>
  </top>
  <left>
    <ListView fx:id="listaSessoes" prefWidth="220"/>
  </left>
  <center>
    <TextArea fx:id="areaDetalhes" editable="false"/>
  </center>
  <bottom>
    <Label text="CineTicket v1.0"/>
  </bottom>
</BorderPane>
```

O comportamento de redimensionamento costuma corresponder ao que se espera: os filhos das posições topo e base recebem a sua altura preferida e esticam-se na largura inteira do **BorderPane**; os filhos de esquerda e direita recebem a sua largura preferida e esticam-se na altura entre o topo e a base; e a posição central recebe todo o espaço que sobrar. Uma posição vazia simplesmente não reserva espaço nenhum — se não houver nada à direita, o centro estende-se até à borda direita da janela.

ATENÇÃO — BorderPane não tem alignment

Ao contrário do **HBox/VBox**, o **BorderPane** não aceita uma propriedade **alignment** própria — não faria sentido, já que a posição de cada filho já é definida pelas cinco áreas. Para alinhar um filho dentro da sua área, use a propriedade anexa **BorderPane.alignment** nesse filho específico.

Layouts aninhados

Quando uma das cinco posições do **BorderPane** precisa de mais do que um controlo — por exemplo, uma fileira de botões na base — a solução é aninhar layouts: colocar um **HBox** dentro do elemento **bottom**. Cada posição do **BorderPane** só aceita um único filho direto, então o aninhamento de layouts é a única

forma de agrupar vários controlos numa mesma área.

StackPane: empilhando camadas

O **StackPane** posiciona todos os seus filhos exatamente na mesma célula, empilhados uns sobre os outros — o primeiro filho declarado fica no fundo da pilha, e o último fica no topo, visível por cima dos demais. Isso é útil, por exemplo, para sobrepor um texto a uma imagem de cartaz, como uma legenda de "ESGOTADO" sobre a pré-visualização de uma sessão:

```
<StackPane>
  <ImageView fitWidth="160" fitHeight="220" image="@cartaz.png"/>
  <Label text="ESGOTADO" style="-fx-background-color: rgba(0,0,0,0.6); -fx-text-fill: white;"/>
</StackPane>
```

06

Controlos Button e Label

Texto, ícones e as propriedades mais usadas nos dois controlos mais comuns

O controlo Button

O **Button** é, de longe, o controlo com que mais se interage numa interface JavaFX. A sua propriedade mais usada é **text**, mas um botão também pode exibir uma imagem através do elemento filho **graphic**, com ou sem texto simultaneamente:

```
<Button text="Reservar bilhete">
  <graphic>
    <ImageView>
      <Image url="@icones/ticket.png"/>
    </ImageView>
  </graphic>
</Button>
```

Vale a pena consultar a documentação da classe **Button** ao menos uma vez: muitas das propriedades disponíveis não pertencem diretamente a **Button**, mas são herdadas de classes na sua hierarquia — **ButtonBase**, **Labeled** e **Control**, entre outras. É por isso que a propriedade **text**, por exemplo, na verdade pertence à classe **Labeled**, compartilhada por todos os controlos que exibem texto, incluindo o **Label**.

O controlo Label

Label mostra texto e, opcionalmente, um gráfico — mas, ao contrário do **Button**, não reage a cliques por padrão. É o controlo indicado para títulos, legendas e qualquer texto estático da interface. Também herda de **Labeled**, por isso partilha propriedades com o **Button**, como cor, tipo de letra e quebra de texto:

```
<Label text="Sessão selecionada" wrapText="true"
  textFill="#B45309">
  <font>
    <Font name="Arial Bold" size="16"/>
  </font>
</Label>
```

A propriedade **wrapText**, quando **true**, faz o texto quebrar em várias linhas em vez de ser cortado quando não há largura suficiente — algo especialmente útil quando o conteúdo de uma etiqueta depende de dados carregados em tempo de execução, como a sinopse de um filme.

REGRA — Alignment em Label só age se houver espaço vazio

A propriedade **alignment** de um **Label** controla onde o texto fica *dentro* da área reservada para ele. Se o **Label** estiver a ocupar apenas o suficiente para caber o próprio texto, não há espaço vazio e o alinhamento parece não ter efeito nenhum. Ele só se torna visível quando o layout dá ao **Label** mais espaço do que o necessário — por exemplo, quando ele ocupa toda a largura de um **BorderPane**.

07

Seleção Única e Múltipla

RadioButton, CheckBox e ToggleButton, e a diferença entre eles

RadioButton e o ToggleGroup

Um **RadioButton** isolado pode ser marcado ou desmarcado livremente, mas raramente é usado sozinho: o objetivo normal é oferecer ao utilizador várias opções das quais só uma pode estar selecionada de cada vez — como o tipo de assento numa sala de cinema. Para isso, os botões precisam de ser agrupados num **ToggleGroup**. Como **ToggleGroup** não descende de **Node**, não pode ser adicionado diretamente ao grafo de cena; por isso, no FXML, ele é declarado com o elemento **fx:define**:

```
<fx:define>
    <ToggleGroup fx:id="grupoAssento"/>
</fx:define>

<RadioButton text="Plateia" toggleGroup="$grupoAssento" selected="true"/>
<RadioButton text="Balcão" toggleGroup="$grupoAssento"/>
<RadioButton text="VIP" toggleGroup="$grupoAssento"/>
```

O prefixo **\$** antes do nome faz referência ao objeto definido em **fx:define** — é a mesma sintaxe usada sempre que se quer apontar para algo criado com **fx:id**. Note também que só é preciso marcar **selected="true"** no botão que deve vir selecionado por padrão; o **ToggleGroup** cuida sozinho de desmarcar os demais quando outro é escolhido.

CheckBox: mais de uma escolha ao mesmo tempo

Quando o utilizador pode selecionar mais do que uma opção simultaneamente — por exemplo, extras da sessão como pipoca e bebida — usa-se **CheckBox** em vez de **RadioButton**. Como **CheckBox** não descende de **ToggleButton**, não é possível agrupá-las num **ToggleGroup**; se for necessário agrupá-las apenas visualmente, basta colocá-las dentro de um **VBox** ou **HBox**:

```
<VBox spacing="6">
    <CheckBox text="Pipoca grande"/>
    <CheckBox text="Refrigerante"/>
    <CheckBox text="Óculos 3D"/>
</VBox>
```

Uma **CheckBox** também suporta um terceiro estado, indeterminado, representado por um traço em vez de uma marca. Esse estado só existe quando **allowIndeterminate** é ativado, e o utilizador não consegue voltar a ele apenas clicando — teria de ser repostado por código. Na prática, é raro haver uma boa razão para usar o estado indeterminado numa aplicação comum.

ToggleButton

Um **ToggleButton** parece um botão normal, mas fica "pressionado" ao ser clicado, e volta ao normal só quando clicado de novo — funciona como um interruptor. Como descende da mesma classe

ToggleButton de onde vem o **RadioButton**, também pode ser agrupado num **ToggleGroup** quando se quer permitir apenas uma seleção de cada vez, mas com a aparência de botão em vez da aparência de círculo de rádio.

08

Campos de Entrada

TextField, PasswordField, ComboBox e ChoiceBox

TextField e PasswordField

O **TextField** é o controlo padrão para entrada de texto de uma única linha — nome do cliente, número de bilhetes, e assim por diante. Além da propriedade **text**, a classe oferece métodos prontos como **copy**, **cut**, **paste** e **selectAll**, o que evita ter de implementar essas ações manualmente.

```
<TextField fx:id="campoNomeCliente" promptText="Nome do cliente"/>
```

PasswordField estende diretamente **TextField** e tem exatamente o mesmo comportamento, com uma única diferença visual: o conteúdo digitado aparece mascarado. Por essa mesma razão, os seus métodos de copiar e cortar não fazem nada — permitir copiar uma senha para a área de transferência anularia o propósito de a mascarar.

ComboBox: escolher uma entre muitas opções

Quando há muitas opções para mostrar como **RadioButton**, um **ComboBox** é a escolha mais económica em espaço: ocupa aproximadamente a mesma área de um **TextField** até o utilizador clicar para revelar a lista completa. A lista de opções é preenchida com **FXCollections.observableArrayList**:

```
<ComboBox fx:id="comboSala" promptText="Escolha a sala">
  <items>
    <FXCollections fx:factory="observableArrayList">
      <String fx:value="Sala 1"/>
      <String fx:value="Sala 2"/>
      <String fx:value="Sala 3 – IMAX"/>
    </FXCollections>
  </items>
  <value><String fx:value="Sala 1"/></value>
</ComboBox>
```

O elemento **value** define qual opção vem pré-selecionada quando a tela abre. É também possível tornar o **ComboBox** editável com **editable="true"**, permitindo ao utilizador digitar um valor que não está na lista — um recurso pouco comum e que costuma confundir mais do que ajudar, por isso vale a pena reservá-lo para casos onde faz sentido real.

ChoiceBox: o primo mais simples

O **ChoiceBox** funciona de forma muito semelhante ao **ComboBox** — também só permite escolher uma opção de cada vez, e também é preenchido com uma lista observável — mas recomenda-se para um número pequeno de itens, enquanto o **ComboBox** lida melhor com listas maiores. Na prática, a diferença de

uso é discreta: o **ChoiceBox** marca a opção selecionada com uma marca de verificação, e o **ComboBox** oferece recursos extras, como as fábricas de células, que permitem personalizar a aparência de cada item.

Controlo	Seleção	Melhor para
ComboBox	Uma opção	Listas maiores; suporta edição e personalização
ChoiceBox	Uma opção	Listas curtas e simples
RadioButton (grupo)	Uma opção	Poucas opções sempre visíveis na tela
CheckBox	Zero ou mais	Opções independentes entre si

09

TitledPane e Accordion

Painéis retráteis para organizar conteúdo em blocos

TitledPane: conteúdo que se recolhe

Apesar do nome terminar em "Pane", o **TitledPane** não é um layout — é um controlo com um título clicável que pode expandir ou recolher o conteúdo associado. É útil sempre que se quer esconder informação secundária sem removê-la da tela, como as políticas de cancelamento de uma reserva:

```
<TitledPane text="Política de cancelamento" expanded="false">
    <Label text="Reservas podem ser canceladas até 2 horas antes da sessão."
        wrapText="true"/>
</TitledPane>
```

Accordion: um TitledPane de cada vez

Um **Accordion** agrupa vários **TitledPane** e garante que apenas um deles fique expandido de cada vez — ao abrir um painel, qualquer outro que estivesse aberto fecha-se automaticamente. É uma forma natural de organizar secções relacionadas, como diferentes categorias de perguntas frequentes sobre a bilheteira:

```
<Accordion>
    <panes>
        <TitledPane fx:id="painelPagamento" text="Formas de pagamento">
            <Label text="Aceitamos cartão e Multibanco." wrapText="true"/>
        </TitledPane>
        <TitledPane text="Trocas e devoluções">
            <Label text="Bilhetes não são reembolsáveis após o início da sessão." wrapText="true"/>
        </TitledPane>
    </panes>
</Accordion>
```

Para que um dos painéis já apareça expandido quando a aplicação é iniciada, atribui-se um **fx:id** a esse **TitledPane** e referencia-se esse id na propriedade **expandedPane** do próprio **Accordion**:

```
<Accordion expandedPane="$painelPagamento">
    ...
</Accordion>
```

ATENÇÃO — Accordion não gira sozinho

Atualmente não existe uma propriedade para orientar um **Accordion** na horizontal diretamente pelo FXML — ele é sempre vertical por padrão. Contornar isso exige rodar o controle por código, uma técnica avançada que foge do escopo desta apostila.

10

Tratamento de Eventos

Como a interface reage a cliques, teclas e mudanças de seleção

Aplicações orientadas a eventos

Uma aplicação de consola é orientada por procedimentos: começa na primeira linha do método **main** e segue, mais ou menos previsivelmente, até à última. Uma aplicação com interface gráfica é diferente — é orientada a eventos. Depois da interface construída, o programa fica à espera, e é o utilizador quem decide qual código será executado a seguir, clicando num botão, digitando num campo ou selecionando um item de uma lista.

Quando o utilizador interage com um controlo, o runtime do JavaFX gera um evento e verifica se alguma parte da aplicação declarou interesse em tratá-lo. Se houver, o manipulador de eventos (também chamado de "ouvinte", do inglês **listener**) associado a esse controlo é executado. Como o controlador é a peça da aplicação responsável por tratar a entrada do utilizador, é lá que os manipuladores de eventos costumam viver.

Ligando um manipulador de eventos no FXML

```
<!-- main.fxml -->
<Button text="Reservar" onAction="#reservarBilhete"/>
```

```
// MainController.java
@FXML
private void reservarBilhete(ActionEvent evento) {
    System.out.println("Bilhete reservado!");
}
```

A propriedade **onAction**, prefixada com **#**, aponta para um método do controlador com o mesmo nome. Esse método precisa de ser anotado com **@FXML** para que o carregador de FXML consiga localizá-lo mesmo sendo privado. O parâmetro **ActionEvent**, embora opcional, é útil quando o mesmo manipulador trata mais de um controlo — através de **evento.getSource()** é possível descobrir qual controlo específico gerou o evento:

```
@FXML
private void reservarBilhete(ActionEvent evento) {
    if (evento.getSource().equals(botaoPlateia)) {
        // lógica para assento de plateia
    } else if (evento.getSource().equals(botaoVip)) {
        // lógica para assento vip
    }
}
```

Ligando um controlo do FXML ao controlador

Para ler ou alterar em código o estado de um controlo declarado no FXML, associa-se um **fx:id** a esse controlo, e declara-se no controlador uma variável de instância com exatamente o mesmo nome, anotada com **@FXML**:

```
<TextField fx:id="campoNomeCliente"/>
```

```
@FXML
private TextField campoNomeCliente;

@FXML
private void reservarBilhete(ActionEvent evento) {
    String nome = campoNomeCliente.getText();
    System.out.println("Reserva em nome de " + nome);
}
```

ATENÇÃO — O nome tem de bater exatamente

O runtime do JavaFX faz essa injeção comparando o valor de **fx:id** com o nome exato da variável Java — maiúsculas, minúsculas e tudo o mais. Um nome divergente não gera erro de compilação, mas resulta numa **NullPointerException** em tempo de execução assim que o controlo for usado.

O método initialize

Sempre que o runtime constrói o controlador a partir do FXML, ele procura um método chamado **initialize**, sem parâmetros, anotado com **@FXML**, e chama-o automaticamente depois de injetar todos os campos anotados. É o lugar certo para preparar o estado inicial da tela — por exemplo, desativar um botão de reserva até que um assento tenha sido escolhido:

```
@FXML
public void initialize() {
    botaoReservar.setDisable(true);
}
```

11

A Thread da Interface Gráfica

Por que tarefas demoradas não podem travar a UI, e como usar `Platform.runLater`

A thread da aplicação JavaFX

O JavaFX executa toda a interface — incluindo os manipuladores de eventos — numa única thread, conhecida como thread da aplicação JavaFX ou, simplesmente, thread da UI. Enquanto um manipulador de eventos estiver a correr, essa thread está ocupada, e a interface deixa de responder a qualquer interação do utilizador. Se esse manipulador demorar vários segundos — por exemplo, a consultar um serviço externo de pagamento — a janela parece travada.

```
@FXML
private void reservarBilhete(ActionEvent evento) {
    try {
        Thread.sleep(8000); // simula uma chamada demorada — NÃO faça isto
    } catch (InterruptedException e) { }
    labelStatus.setText("Reserva confirmada!");
}
```

Executar esse código diretamente no manipulador de eventos congela a janela pelos 8 segundos inteiros: nenhum botão responde, e em alguns sistemas operativos a janela chega a ser marcada como "não está a responder".

Threads em segundo plano

A solução é mover o trabalho demorado para uma thread separada, chamada thread de fundo, deixando a thread da UI livre para continuar a atender o utilizador enquanto o processamento decorre em paralelo:

```
@FXML
private void reservarBilhete(ActionEvent evento) {
    Runnable tarefa = () -> {
        try {
            Thread.sleep(8000);
        } catch (InterruptedException e) { }
        // ainda não podemos tocar na UI aqui!
    };
    new Thread(tarefa).start();
}
```

ATENÇÃO — O grafo de cena não é seguro para threads

Tentar atualizar diretamente um controlo (por exemplo, `labelStatus.setText(...)`) a partir de uma thread de fundo lança uma **`IllegalStateException`**, com a mensagem "Not on FX application thread". O grafo de cena assume que só a thread da UI o altera; permitir que várias threads o façam ao mesmo tempo poderia corromper o estado interno dos nós.

Voltando à thread da UI com `Platform.runLater`

Para atualizar a interface depois de a thread de fundo terminar o seu trabalho, usa-se

`Platform.runLater`, que agenda um **`Runnable`** para ser executado na thread da UI assim que ela ficar livre:

```
@FXML
private void reservarBilhete(ActionEvent evento) {
    Runnable tarefa = () -> {
        try {
            Thread.sleep(8000);
        } catch (InterruptedException e) { }

        Platform.runLater(() -> labelStatus.setText("Reserva confirmada!"));
    };
    new Thread(tarefa).start();
}
```

Com esse ajuste, a janela continua responsiva durante os 8 segundos de espera simulada, e a etiqueta de estado só é atualizada — de forma segura, dentro da thread da UI — quando o trabalho em segundo plano termina. O JavaFX também disponibiliza, no pacote **`javafx.concurrent`**, classes como **`Task`** que encapsulam esse padrão de forma mais estruturada, mas **`Platform.runLater`** já resolve os casos mais simples.

12

Modelo de Dados e ListView

Construindo um catálogo de sessões com uma classe de modelo e uma lista visual

A classe de modelo Sessao

Antes de preencher qualquer lista visual, é preciso decidir que forma os dados vão ter. Uma sessão de cinema, no CineTicket, tem um filme, uma sala e uma data/hora — informação suficiente para uma classe de modelo simples:

```
public class Sessao {
    private String filme;
    private String sala;
    private LocalDateTime horario;

    public Sessao(String filme, String sala, LocalDateTime horario) {
        this.filme = filme;
        this.sala = sala;
        this.horario = horario;
    }

    public String getFilme() { return filme; }
    public String getSala() { return sala; }
    public LocalDateTime getHorario() { return horario; }

    @Override
    public String toString() {
        return filme;
    }
}
```

A sobrescrita de **toString** é intencional: como veremos a seguir, um **ListView** exibe cada item chamando exatamente esse método, a menos que se personalize a exibição de outra forma. Devolver apenas o nome do filme garante que a lista mostre um rótulo curto e legível.

Preenchendo o ListView

O **ListView** é o controlo indicado para mostrar uma coleção de itens. Primeiro, atribui-se um **fx:id** ao controlo no FXML; depois, no controlador, os dados são carregados dentro de uma **ObservableList**:

```
<ListView fx:id="listaSessoes" prefWidth="240"/>
```

```
@FXML
private ListView<Sessao> listaSessoes;

private List<Sessao> sessoes;

@FXML
public void initialize() {
    sessoes = new ArrayList<>();
    sessoes.add(new Sessao("Duna: Parte Três", "Sala 1",
        LocalDateTime.of(2026, 9, 12, 19, 0)));
    sessoes.add(new Sessao("Viagem ao Fundo do Mar", "Sala 2",
        LocalDateTime.of(2026, 9, 12, 16, 30)));

    listaSessoes.getItems().setAll(sessoes);
    listaSessoes.getSelectionModel().setSelectionMode(SelectionMode.SINGLE);
}
```

Declarar o **ListView** com o tipo genérico **Sessao** (**ListView<Sessao>**) evita conversões manuais de tipo mais adiante, quando formos ler o item selecionado. **SelectionMode.SINGLE** garante que apenas um item pode estar selecionado de cada vez — a escolha certa quando a área de detalhes só mostra informação de uma sessão por vez.

Reagindo à seleção

Em vez de tratar apenas o clique do rato, é mais robusto ouvir diretamente as mudanças na propriedade **selectedItem** do modelo de seleção — assim, tanto um clique do utilizador quanto uma seleção feita por código (como seleccionar o primeiro item ao abrir a aplicação) disparam o mesmo tratamento:

```
listaSessoes.getSelectionModel().selectedItemProperty()
    .addListener((observable, valorAntigo, valorNovo) -> {
        if (valorNovo != null) {
            areaDetalhes.setText(valorNovo.getSala() + "\n" + valorNovo.getHorario());
        }
    });

listaSessoes.getSelectionModel().selectFirst();
```

13

Persistência em Ficheiro de Texto

Carregando e gravando dados com um singleton e `java.nio.file`

Por que não usar apenas dados em memória

Sem persistência, cada vez que o `CineTicket` é reiniciado, todas as sessões cadastradas pelo utilizador desapareceriam — a aplicação voltaria a mostrar apenas os dados de exemplo. Para uma aplicação de exemplo, um ficheiro de texto simples, com um valor por coluna separado por tabulação, é suficiente; aplicações reais costumam preferir XML, JSON ou uma base de dados, sobretudo quando o volume de registos cresce.

Um repositório em singleton

Tanto a classe principal (que precisa de gravar os dados ao fechar) quanto o controlador (que precisa de os carregar ao abrir) precisam de acesso aos mesmos dados. Uma forma conveniente de partilhar esse acesso é um singleton: uma classe que garante a existência de uma única instância durante toda a execução da aplicação, com um construtor privado e um método estático de acesso:

```
public class SessaoRepositorio {
    private static SessaoRepositorio instancia = new SessaoRepositorio();
    private static final String FICHEIRO = "sessoes.txt";

    private List<Sessao> sessoes;
    private DateTimeFormatter formato;

    private SessaoRepositorio() {
        formato = DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm");
    }

    public static SessaoRepositorio getInstance() {
        return instancia;
    }

    public List<Sessao> getSessoes() { return sessoes; }
}
```

O construtor privado impede que qualquer outra classe crie uma nova instância com `new SessaoRepositorio()`; o único caminho de acesso é `SessaoRepositorio.getInstance()`, que devolve sempre o mesmo objeto.

Carregando e gravando com `java.nio.file`

O pacote `java.nio.file` fornece uma API mais moderna do que a antiga `java.io` para trabalhar com ficheiros, através das classes `Path` e `Files`:

```
public void carregarSessoes() throws IOException {
    List<Sessao> lista = new ArrayList<>();
    Path caminho = Paths.get(FICHEIRO);

    try (BufferedReader br = Files.newBufferedReader(caminho)) {
        String linha;
        while ((linha = br.readLine()) != null) {
            String[] partes = linha.split("\\t");
            LocalDateTime horario = LocalDateTime.parse(partes[2], formato);
            lista.add(new Sessao(partes[0], partes[1], horario));
        }
    }
    sessoes = lista;
}

public void gravarSessoes() throws IOException {
    Path caminho = Paths.get(FICHEIRO);
    try (BufferedWriter bw = Files.newBufferedWriter(caminho)) {
        for (Sessao s : sessoes) {
            bw.write(String.format("%s\\t%s\\t%s",
                                   s.getFilme(), s.getSala(), s.getHorario().format(formato)));
            bw.newLine();
        }
    }
}
```

O uso de **try** com recursos garante que o **BufferedReader/BufferedWriter** é fechado automaticamente, mesmo que ocorra uma exceção a meio da leitura ou da escrita.

Ligando aos pontos certos do ciclo de vida

carregarSessoes deve ser chamado quando o controlador é inicializado, e **gravarSessoes** deve ser chamado no método **stop** da classe **Application** — o ponto do ciclo de vida executado quando o utilizador fecha a janela principal:

```
@Override
public void stop() throws IOException {
    SessaoRepositorio.getInstance().gravarSessoes();
}
```

14

Ordenação e Filtragem

SortedList e FilteredList para organizar e esconder itens sem reescrever a lista

SortedList: ordenar sem reescrever a lista

Em vez de ordenar manualmente a **ObservableList** sempre que um item é adicionado ou removido, o JavaFX oferece a classe **SortedList**, que envolve uma **ObservableList** existente e mantém a ordenação automaticamente, aplicando o **Comparator** fornecido a cada mudança:

```
ObservableList<Sessao> dados = FXCollections.observableArrayList(
    SessaoRepositorio.getInstance().getSesoes());

SortedList<Sessao> sesoesOrdenadas = new SortedList<>(dados, new Comparator<Sessao>() {
    @Override
    public int compare(Sessao s1, Sessao s2) {
        return s1.getHorario().compareTo(s2.getHorario());
    }
});

listaSesoes.setItems(sesoesOrdenadas);
```

O método **compare** deve devolver um valor negativo quando o primeiro argumento é considerado menor, zero quando são equivalentes, e um valor positivo quando o primeiro é considerado maior. Como **LocalDateTime** já implementa **Comparable**, basta delegar a comparação ao seu próprio método **compareTo**. A partir desse ponto, adicionar ou remover sessões da **ObservableList** original mantém a lista visível sempre ordenada, sem qualquer código extra.

FilteredList: esconder itens sem os apagar

A **FilteredList** funciona de forma parecida, mas em vez de ordenar, decide quais itens aparecem e quais ficam escondidos, com base num **Predicate** — uma interface funcional com um único método, **test**, que devolve **true** para manter o item e **false** para excluí-lo da vista:

```
FilteredList<Sessao> sesoesFiltradas = new FilteredList<>(dados, s -> true);

// mostrar só as sessões de hoje
sesoesFiltradas.setPredicate(s ->
    s.getHorario().toLocalDate().equals(LocalDate.now()));

// voltar a mostrar todas
sesoesFiltradas.setPredicate(s -> true);
```

É possível — e comum — combinar as duas classes: primeiro filtrar a lista original, e depois ordenar o que sobrou, envolvendo a **FilteredList** dentro de uma **SortedList**:

```
SortedList<Sessao> resultado = new SortedList<>(sessoesFiltradas, comparadorPorHorario);  
listaSessoes.setItems(resultado);
```

DICA — Reagindo à troca de filtro

Ao aplicar um novo predicado, a seleção da lista costuma ser perdida, já que o item antes selecionado pode ter deixado de fazer parte do conjunto filtrado. Vale testar se **`sessoesFiltradas.isEmpty()`** antes de tentar selecionar algo, e cair de volta para **`selectFirst()`** quando o item anteriormente selecionado já não estiver presente.

15

Estilização com CSS

Seletores de classe e de id, folhas de estilo externas e temas do JavaFX

CSS em linha

Assim como o HTML, o JavaFX aceita estilos CSS diretamente num controlo através da propriedade **style**, com propriedades prefixadas por **-fx-**:

```
<Button text="Reservar" style="-fx-background-color: #B45309; -fx-text-fill: white;"/>
```

Isso funciona, mas não escala: se a aplicação tiver dezenas de botões espalhados por vários ficheiros FXML e, mais tarde, alguém decidir trocar a cor de todos eles, seria preciso encontrar e editar cada botão individualmente.

Folhas de estilo externas

A alternativa recomendada é reunir os estilos num ficheiro **.css** separado, associado ao FXML através da propriedade **stylesheets**, aplicada ao nó raiz:

```
/* styles.css */
.button {
    -fx-background-color: #B45309;
    -fx-text-fill: white;
}

#botaoCancelar {
    -fx-background-color: #6B7280;
}
```

```
<BorderPane stylesheets="@styles.css" ...>
    <Button text="Reservar"/>
    <Button fx:id="botaoCancelar" id="botaoCancelar" text="Cancelar"/>
</BorderPane>
```

O seletor **.button**, com ponto, atinge todas as instâncias da classe **Button** da aplicação — o equivalente ao seletor de classe CSS na web. Já o seletor **#botaoCancelar**, com cardinal, atinge apenas o controlo cujo atributo **id** (não confundir com **fx:id**, usado para referenciar o controlo em código) corresponde a esse valor.

Ordem de precedência

Quando o mesmo controlo recebe estilo de mais de uma fonte, o JavaFX aplica a seguinte ordem de precedência, da mais forte para a mais fraca:

Ordem	Fonte do estilo
1ª	CSS em linha, na própria tag do controle (style="...")
2ª	Folha de estilos associada ao FXML (stylesheets)
3ª	Folha de estilos do tema da aplicação (Modena, por padrão)

Trocando o tema da aplicação

O JavaFX aplica, por padrão, uma folha de estilos chamada Modena a toda a aplicação. É possível trocar esse tema global — o que afeta todas as janelas e diálogos, sem precisar de adicionar **stylesheets** a cada FXML — chamando **setUserAgentStylesheet** logo no arranque da aplicação:

```
@Override
public void start(Stage stage) throws Exception {
    Application.setUserAgentStylesheet(Application.STYLESHEET_CASPIAN);
    // ... resto do start
}
```

DICA — Consultando a documentação de CSS do JavaFX

A referência oficial de CSS do JavaFX lista todos os seletores e propriedades disponíveis para cada controle, incluindo layouts. Vale a pena consultá-la sempre que precisar estilizar um controle menos comum, em vez de tentar adivinhar o nome da propriedade **-fx-...**

16

Transformações, Efeitos e Diálogos

Zoom, rotação, sombras e as classes FileChooser e DirectoryChooser

Zoom e rotação

Qualquer nó do grafo de cena pode ser transformado com métodos como **setScaleX**, **setScaleY** e **setRotate**. Combinado com manipuladores de eventos de rato, é fácil criar um pequeno efeito de destaque ao passar o cursor sobre o cartaz de um filme:

```
<ImageView fx:id="cartazFilme" fitWidth="140"
           onMouseEntered="#ampliarCartaz"
           onMouseExited="#restaurarCartaz"/>

@FXML
private void ampliarCartaz(MouseEvent evento) {
    cartazFilme.setScaleX(1.15);
    cartazFilme.setScaleY(1.15);
}

@FXML
private void restaurarCartaz(MouseEvent evento) {
    cartazFilme.setScaleX(1.0);
    cartazFilme.setScaleY(1.0);
}
```

Efeitos visuais

O pacote **javafx.scene.effect** oferece efeitos prontos, como sombras, desfoque e brilho, aplicados a qualquer nó através do método **setEffect**. Como o suporte a efeitos no FXML ainda é limitado, a forma mais confiável de os aplicar é por código, geralmente dentro de **initialize**:

```
@FXML
private Button botaoDestaque;

@FXML
public void initialize() {
    botaoDestaque.setEffect(new DropShadow());
}
```

A classe **DropShadow** aceita propriedades como cor, raio e desvio, permitindo ajustar a intensidade e a direção da sombra conforme necessário.

FileChooser: abrir e gravar ficheiros

A classe **FileChooser** disponibiliza uma caixa de diálogo nativa de abrir/gravar ficheiro, sem que seja preciso construir essa interface manualmente. Ela não descende de **Node**, por isso nunca é adicionada a um layout — é apenas instanciada e exibida quando necessária:

```
@FXML
private void importarSessoes(ActionEvent evento) {
    FileChooser seletor = new FileChooser();
    seletor.setTitle("Importar lista de sessões");

    File ficheiro = seletor.showOpenDialog(painelRaiz.getScene().getWindow());
    if (ficheiro != null) {
        // processar o ficheiro escolhido
    }
}
```

O parâmetro passado a **showOpenDialog** é a janela-pai da caixa de diálogo — geralmente obtida a partir de qualquer nó já presente na cena, através de **no.getScene().getWindow()**. Isso torna a caixa de diálogo modal: enquanto ela estiver aberta, a janela principal fica bloqueada, o que evita que o utilizador acabe com várias instâncias do seletor abertas ao mesmo tempo.

ATENÇÃO — Escolher uma pasta é diferente de escolher um ficheiro

O **FileChooser** deixa o utilizador entrar numa pasta e escolher um ficheiro lá dentro — não é possível usá-lo para seleccionar apenas a pasta em si. Quando esse for o objetivo (por exemplo, escolher onde exportar um relatório de sessões), a classe correta é **DirectoryChooser**, que se usa de forma muito semelhante:

```
DirectoryChooser seletorPasta = new DirectoryChooser();
File pasta = seletorPasta.showDialog(painelRaiz.getScene().getWindow());
```

Fim de apostila

Percorremos os fundamentos do JavaFX: a anatomia de uma aplicação, os principais layouts, os controlos mais usados, o tratamento de eventos, a thread da interface gráfica, a construção de um modelo de dados com ListView, a persistência simples em ficheiro, a ordenação e filtragem com SortedList e FilteredList, a estilização com CSS, e transformações, efeitos e diálogos de ficheiro.

Estes fundamentos sustentam qualquer aplicação desktop maior em Java — a partir daqui, o caminho natural é aprofundar tópicos como binding de propriedades, tabelas (TableView), gráficos (Chart) e a integração do JavaFX com bases de dados e serviços externos.

JavaStudiesHub